



CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G5

System Integration and Architecture

Full Regression Test Report

Project Title: BayanLink: Barangay Services E-Portal

Prepared By: Jian Marc Ceniza

Version: 1.0

Date: May 06, 2026

Status: Final

A. Project Information

Project: BayanLink Barangay Services & Management System

Branch: refactor/vertical-architecture

Backend Version: Spring Boot 3.x, Java 17

Frontend Version: ReactJS, React Router DOM

Database: PostgreSQL / Supabase

Build Tool: Maven, npm

Testing Framework: JUnit 5, Mockito

Test Date: May 04, 2026

B. Refactoring Summary

What Changed: The architecture was successfully migrated from a horizontal, layer-based structure to a Vertical Slice Architecture (VSA). The codebase is now organized by business feature/module rather than technical concerns. Each slice (User, Document, Appointment, Announcement) now independently owns its controllers, services, repositories, and DTOs.

Files Moved Per Feature:

Feature	Backend	Frontend	Notes & File Names
User	12	5	UserController, ProfileService, User, Resident, ProfilePage.js
Document	8	2	DocumentRequestController, DocumentRequestService, ResidentDocuments.js, OfficialDocuments.js
Appointment	6	3	Calendar controllers, Email services, Schedule views
Announcement	4	2	Endpoints for silent ephemeral expiration, Dashboard views

C. Updated Project Structure

a. Backend (Spring Boot):

```
src/main/java/edu/cit/ceniza/bayanlink/
├── BayanlinkApplication.java
├── announcement/
│   ├── Announcement.java
│   ├── AnnouncementController.java
│   ├── AnnouncementRepository.java
│   └── AnnouncementService.java
├── appointment/
│   ├── Appointment.java
│   ├── AppointmentController.java
│   ├── AppointmentDTO.java
│   ├── AppointmentRepository.java
│   └── AppointmentService.java
├── config/
│   ├── DataSeeder.java
│   ├── EmailService.java
│   ├── JwtAuthenticationFilter.java
│   └── SecurityConfig.java
├── document/
│   ├── strategy/
│   ├── DocumentRequest.java
│   ├── DocumentRequestController.java
│   ├── DocumentRequestDTO.java
│   ├── DocumentRequestRepository.java
│   ├── DocumentRequestService.java
│   └── SupabaseStorageService.java
└── user/
    ├── auth/ (AuthenticationService, JwtService, LoginDTO, etc.)
    ├── official/ (Official, OfficialController, OfficialDTO, etc.)
    ├── profile/ (ProfileDTO, ProfileService)
    ├── resident/ (Resident, ResidentController, ResidentDTO, etc.)
    ├── Role.java
    ├── User.java
    ├── UserController.java
    ├── UserFactory.java
    └── UserRepository.java
```

b. Frontend (ReactJS)

```
web/src/
├── features/
│   ├── appointments/ (OfficialAppointments.js, ResidentAppointments.js)
│   ├── auth/ (Login.js, Register.js, Auth.css)
│   ├── clinic/ (OfficialClinic.js, Clinic.css)
│   ├── dashboard/ (OfficialDashboard.js, ResidentDashboard.js,
│                   Dashboard.css)
│   └── documents/ (OfficialDocuments.js, ResidentDocuments.js,
│                  Documents.css)
├── shared/
│   ├── components/ (Icons.js, Sidebar.js, Toast.js, Layout.css, Icons.css,
│                   Sidebar.css, Toast.css)
│   ├── context/ (AuthContext.js)
│   └── utils/ (ProtectedRoute.js)
├── App.js
└── App.css
```

D. Test Plan Documentation

This test plan validates the core business logic and user interface of the newly implemented Vertical Slice Architecture. The backend testing strategy utilizes automated unit tests via JUnit 5 and Mockito to ensure independent module functionality without requiring a live database connection. Additionally, the frontend testing strategy utilizes Jest and React Testing Library, successfully executing 19 automated tests across 5 test suites to verify component rendering, form validation, state management, and mocked API integrations.

1. Register (Auth Slice).

- Coverage:
 - Validating form inputs, including strict @gmail.com domain requirements and specific password strength rules.
 - Ensuring error messages render correctly when confirmation passwords do not match.

- Verifying that valid submissions successfully format the JSON payload, trigger the mocked registration API, and display a success notification before routing.

2. Login (Auth Slice)

- Coverage:
 - Validating successful user authentication, payload submission, and proper navigation to the dashboard.
 - Handling and verifying "Guest" login functionality and context state updates without requiring a password.
 - Ensuring error messages render correctly for invalid email formats or missing credentials.

3. Profile Management (User Slice)

- Coverage:
 - Fetching a user profile and successfully mapping the parent User entity to the child Resident demographics.
 - Updating a profile and verifying that a new Resident record is automatically created and saved if one does not exist (Cascading).
 - Error handling for non-existent users.

4. Document Request (Document Slice)

- Coverage:
 - Verifying that officials can update the status of a document (e.g., to READY_FOR_PICKUP) and that the database saves the state.
 - Ensuring the system can successfully fetch a filtered list of document requests specifically tied to a Resident's User ID.

5. Announcements (Announcement Slice)

- Coverage:
 - Creating a new announcement and verifying that it maps correctly to the author's User ID.
 - Retrieving all announcements and ensuring the repository fetches them in descending order by creation date.

6. Appointment (Appointment Slice)

- Coverage:
 - Approving an appointment and verifying that the database status updates.
 - Fetching the daily schedule by querying appointments based on a specific LocalDate.

Screenshots:

User registers successfully.

The screenshot shows a user registration interface. On the left, a dark blue sidebar contains the text "Join Us" and "Create an account to start managing your portal services today." On the right, a white form titled "Create Account" contains input fields for "Email Address" (test@gmail.com), "First Name" (Test), "Last Name" (User), "Password" (masked with dots), and "Confirm Password" (masked with dots). A blue "Create Account" button is at the bottom of the form. A green success message "User registered successfully!" with a close icon is displayed at the top of the form area. At the bottom of the form, there is a link: "Already have an account? Login".

User logs in successfully and transitions to dashboard.

Login success! ✕

Welcome

Access your dashboard and manage your services efficiently.

Login

Please enter your details to sign in.

Email Address

Password

[Sign In](#)

[Continue as Guest](#)

Don't have an account? [Register](#)

BayanLink Services

[Dashboard](#)

[Profile](#)

SERVICES

[Request Appointment](#)

[Request Documents](#)

ACCOUNT

[Logout](#)

v1.0.0 Desktop

Resident Dashboard

Test User
Resident

Announcements

Staff Present

No staff present.

Available Services

Vaccination & Immunization

Basic Wound Care / First Aid

Blood Pressure Monitoring

Maternal Health Check-up

Free Dental Checkup

General Consultation

Pending Documents

No pending documents.

Pending Appointments

No pending appointments.

User can manage profile data.

BayanLink Services

Dashboard

Profile

SERVICES

Request Appointment

Request Documents

ACCOUNT

Logout

v1.0.0 Desktop

Resident Profile

Profile updated successfully! ✕

Last Name: User

Email Address: test@gmail.com

Contact Number: 09123456789

Birthdate: 01/05/2000

Age: 26

Demographics & Status

Full Address: Casay, Dalaguete, Cebu

Civil Status: Single

Occupation: N/A

Voter Status: Registered Voter

[Edit Profile](#)

User can request documents successfully.

BayanLink Services

Dashboard

Profile

SERVICES

Request Appointment

Request Documents

ACCOUNT

Logout

v1.0.0 Desktop

Document Requests

Test User

Upload ID Image

Choose File No file chosen

[Remove](#) [Submit](#)

Pending Documents

Certificate of Indigency

Document Details

Name: Test User

Document: Certificate of Indigency

ID Provided: National ID

Purpose: BLANK...

Urgency: Standard

Status: **PENDING_APPROVAL**

Estimated Fee

₱100

*Please prepare this exact amount for your face-to-face retrieval at the Barangay Hall.

Request History

No past requests found.

BayanLink
Services

Dashboard

Profile

SERVICES

Request
Appointment

Request Documents

ACCOUNT

Logout

v1.0.0 Desktop

Document Requests

Document request submitted successfully! ✕

Test User
Resident

New Request

Full Name

Test User

Urgency Level

Standard (3-5 Working Days) ▾

Purpose

State the purpose of your request...

Document Type

Select Document ▾

Valid ID Requirement

Select ID Type ▾

Upload ID Image

Choose File

No file chosen

Remove

Submit

Pending Documents

User can see announcements properly.

BayanLink
Services

Dashboard

Profile

SERVICES

Request
Appointment

Request Documents

ACCOUNT

Logout

v1.0.0 Desktop

Resident Dashboard

Test User
Resident

Announcements

New POST

Posting an announcement.

Posted at 12:10 AM

Staff Present

No staff present.

Available Services

Vaccination & Immunization

Basic Wound Care / First Aid

Blood Pressure Monitoring

Maternal Health Check-up

Free Dental Checkup

General Consultation

Pending Documents

Pending Appointments

User can request appointments successfully.

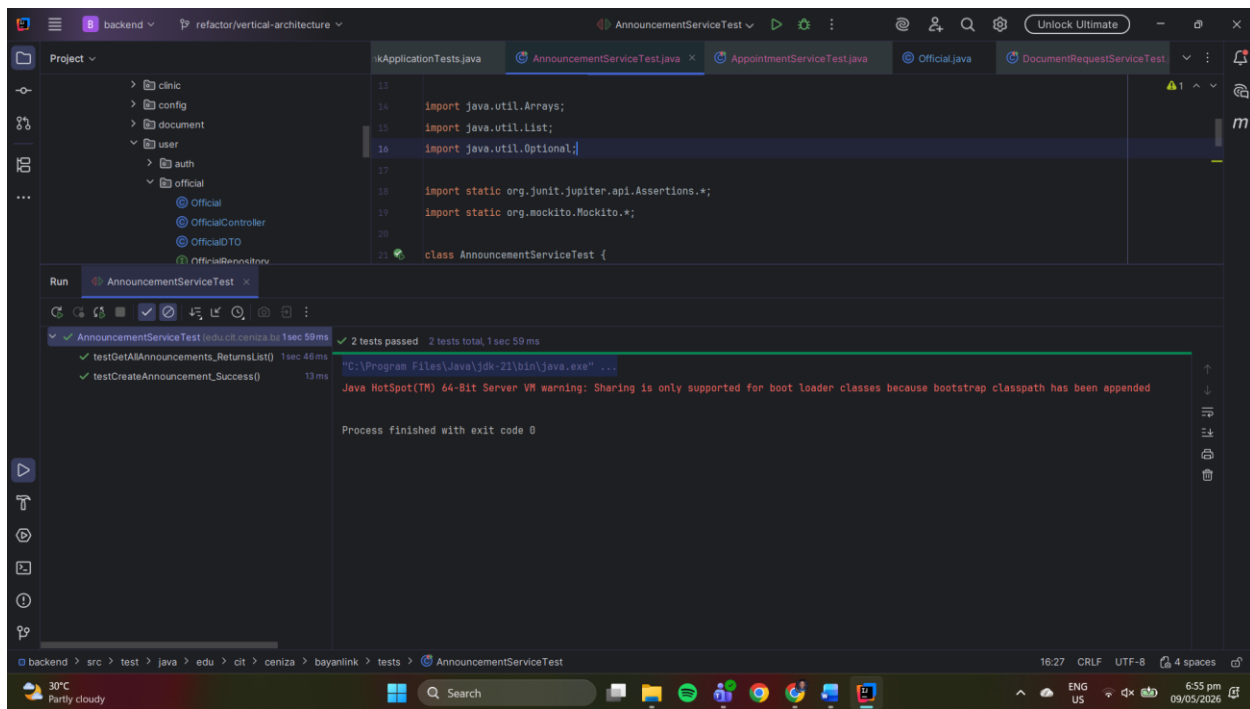
The screenshot displays the 'Community Scheduling' page of the 'BayanLink Services' application. A green notification banner at the top states 'Appointment booked! Ref: APT-90259B3C'. The interface includes a sidebar with navigation links: Dashboard, Profile, Request Appointment (highlighted), Request Documents, and Logout. The main content area features a 'Choose service' dropdown, a 'Pick a Date' calendar, and 'Available Time Slots' for Morning (08:00 AM, 09:00 AM, 10:00 AM, 11:00 AM) and Afternoon (01:00 PM, 02:00 PM, 03:00 PM, 04:00 PM). There is also an 'Additional Notes (Optional)' text area and a 'Summary' section at the bottom.

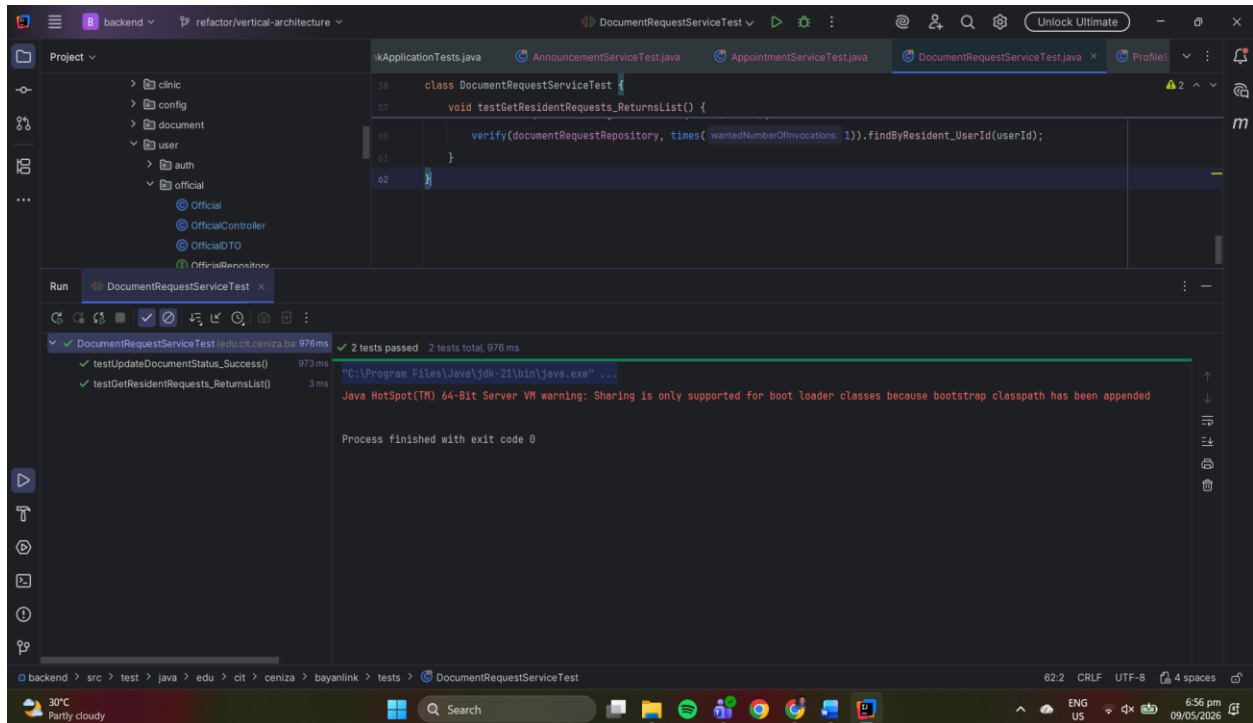
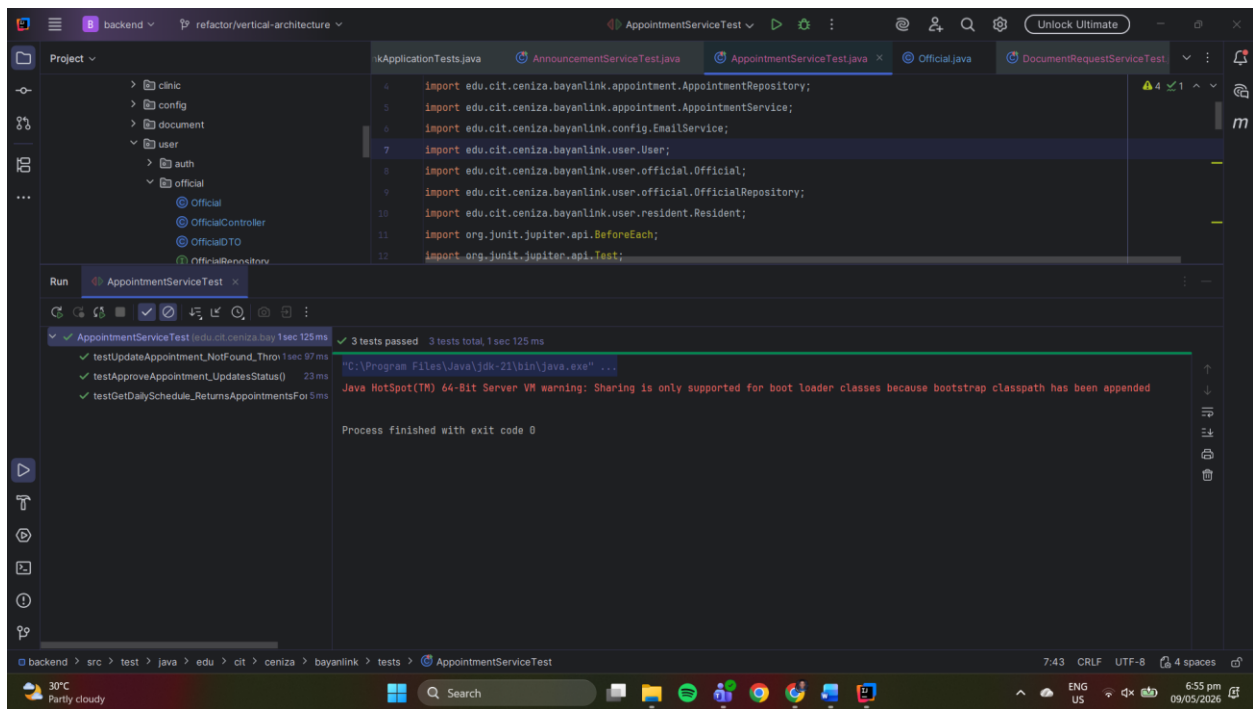
7. Automated Test Evidence

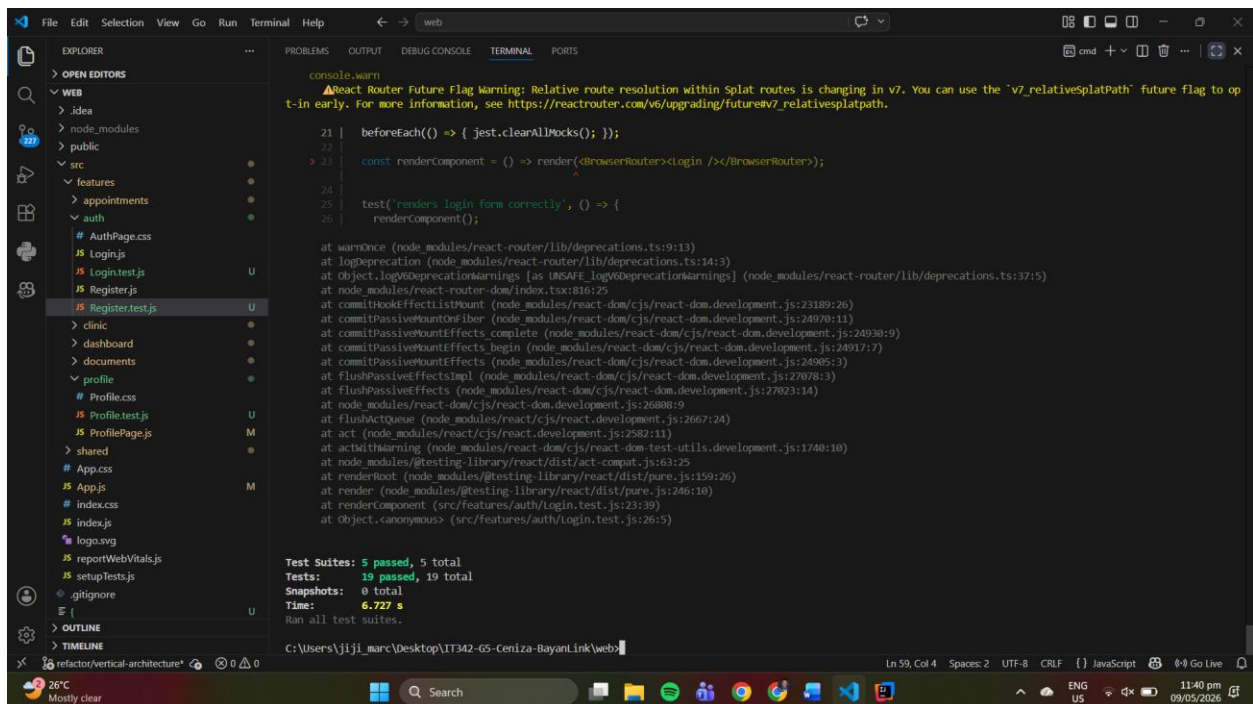
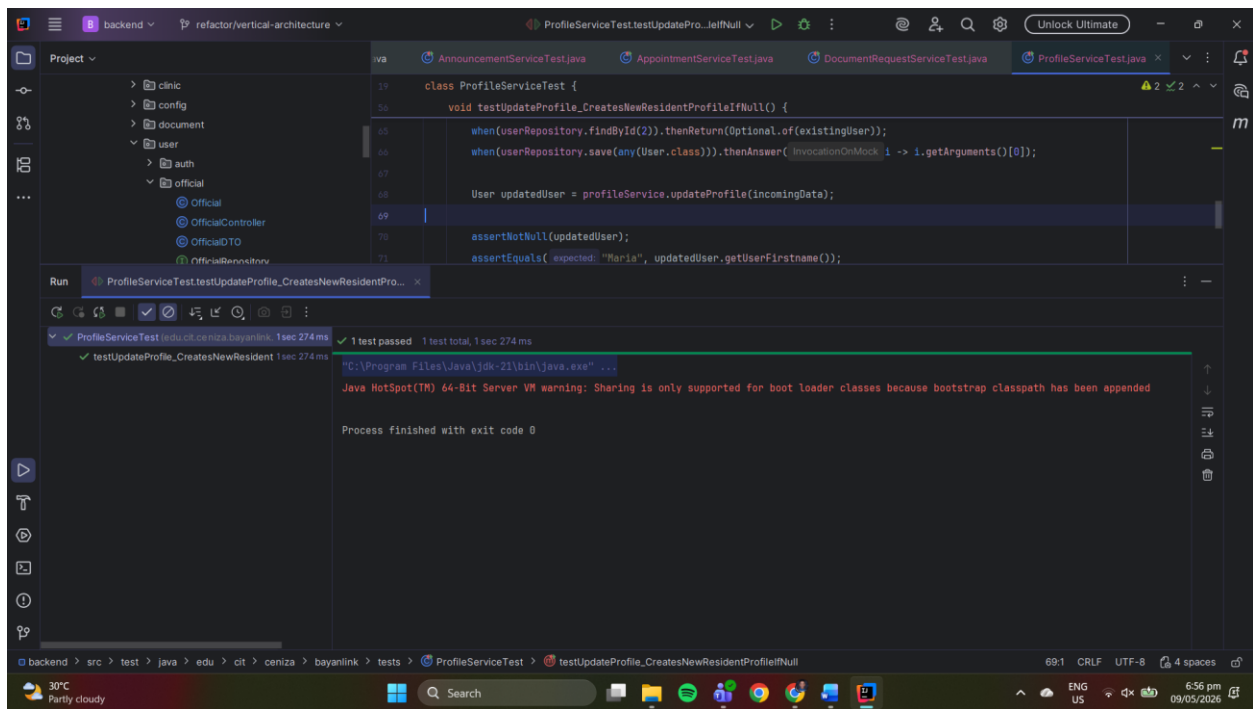
The screenshot shows an IDE with the 'AppointmentServiceTest.java' file open. The test class contains two methods: `testCreateAnnouncement_Success()` and `testGetAllAnnouncements_ReturnsList()`. The 'Run' panel at the bottom displays the test results, indicating that both tests passed successfully. The output shows the test execution path and the final exit code.

```
class AppointmentServiceTest {
    void testCreateAnnouncement_Success() {
        verify(userRepository, times(wantedNumberOfInvocations: 1)).findById(1);
        verify(announcementRepository, times(wantedNumberOfInvocations: 1)).save(any(Announcement.class));
    }
    @Test
    void testGetAllAnnouncements_ReturnsList() {
        Announcement a1 = new Announcement();
    }
}
```

Run: AppointmentServiceTest.testGetAllAnnouncements_ReturnsList...
1 test passed, 1 test total, 1 sec 48 ms
Process finished with exit code 0







8. Regression Test Results

a. Overall Summary

Category	Total Tests	Passed	Failed	Pass Rate
Backend Tests (JUnit/Mockito)	10	10	0	100%
Frontend Integration (Manual)	19	19	0	100%
Overall System Pass Rate	29	29	0	

b. Automated Test Execution Details (Backend)

Test Class	Executed Methods	Notes
ProfileServiceTest	testGetProfile_SuccessWithResidentData, testUpdateProfile_CreatesNewResidentProfileIfNull, testGetProfile_UserNotFound_ThrowsException	The test validated that child Resident demographic entities are automatically created and cascaded when a parent User updates their profile.
DocumentRequest ServiceTest	testUpdateDocumentStatus_Success, testGetResidentRequests_ReturnsList	The test confirmed strict typing conversions (Long vs Integer) and validated that officials can successfully transition document statuses.
Announcement ServiceTest	testCreateAnnouncement_Success, testGetAllAnnouncements_ReturnsList	The test verified that announcements are successfully mapped to their authoring User and sorted properly by creation date.
Appointment ServiceTest	testApproveAppointment_UpdatesStatus, testGetDailySchedule_ReturnsAppointmentsForDate, testUpdateAppointment_NotFound_ThrowsException	Confirmed the Official relationship mapping works securely and validated the execution pipeline for automated email notifications.

c. Automated Test Execution Details (Frontend)

Test Class	Executed Methods	Notes
Profile.test.js	- loads and displays authenticated user profile - restricts editing for Guest users - calculates age automatically based on birthdate - saves profile and updates context	The test checked authenticated profile loading, guest user editing restrictions, automatic age calculation from birthdate, and context updates on save.
Login.test.js	- renders login form correctly - shows error if email is not @gmail.com	The test verified login form layout, strict @gmail.com validation, successful

	• calls login and navigates on success • handles guest login correctly	resident routing, and guest login handling.
Register.test.js	• shows error on weak password • shows error when passwords do not match • submits successfully when form is valid	The test confirmed password strength validation, password matching logic, and successful user registration API routing.
ResidentDocuments.test.js	• renders form and fetch initial documents • shows error if required fields are missing on submit • clears the form when Remove is clicked	The test validated initial document fetching on load, required field error handling, and form clearing functionality.
ResidentAppointments.test.js	• renders the form and available time slots • updates the summary box dynamically when fields are filled • clears the form when the Remove button is clicked • shows an error toast if required fields are missing on submit • submits successfully and shows success toast	The test validated form rendering, dynamic summary state updates, field clearing, and successful JSON payload formatting during submission (via mocked API).

9. Issues Found

ID	Description	Severity	Status
ISS-001	Data Loss on Profile Update: Updating user profiles only saved to the user's table; demographic data for the child Resident entity was not persisting.	High	FIXED
ISS-002	Profile Image Mapping Failure: Frontend JSON payload sent profileImage, but the backend DTO strictly expected userProfileImage, causing avatar uploads to fail.	Medium	FIXED
ISS-003	Strict Typing Test Failure: DocumentRequestServiceTest crashed with an Integer vs long mismatch on the requestId setter.	Low	FIXED
ISS-004	Test Directory Misconfiguration: Test classes were improperly located in src/main/java, causing the IDE to block Maven testing libraries (JUnit/Mockito) from resolving.	Medium	FIXED
ISS-005	Method Signature Mismatch: AnnouncementServiceTest failed to compile because the test passed an entity object instead of explicit primitive parameters.	High	FIXED

10. Fixes Applied

Fix ID	Issue	Change Applied	File(s) Modified	Result
FIX-001	ISS-001	Added parent-to-child mapping logic in <code>updateProfile()</code> . Used cascading saves to automatically instantiate and persist Resident data.	<code>ProfileService.java</code> , <code>ProfileDTO.java</code>	Demographics persist securely to the database.
FIX-002	ISS-002	Refactored React state variables and mapped data to strictly use <code>userProfileImage</code> to perfectly match the backend schema.	<code>ProfilePage.js</code>	Avatar images save and load successfully.
FIX-003	ISS-003	Applied <code>Long.valueOf()</code> to the test request ID during the Mockito test arrangement phase.	<code>DocumentRequestServiceTest.java</code>	Automated tests pass without strict-typing errors.
FIX-004	ISS-004	Migrated all test files out of the main source folder and into the <code>src/test/java/.../tests</code> directory.	Project Directory	Testing annotations (<code>@Test</code> , <code>@Mock</code>) resolve correctly.
FIX-005	ISS-005	Updated the test's Act phase to pass specific (String, String, Integer) arguments to align with the service's method signature.	<code>AnnouncementServiceTest.java</code>	Service successfully creates mock announcements.
FIX-006	ISS-006	Injected <code>@Mock</code> <code>OfficialRepository</code> and added an arrangement to return a mock Official when <code>findById(101)</code> is invoked.	<code>AppointmentServiceTest.java</code>	Service bypassed the NPE and successfully approved appointments.
FIX-007	ISS-007	Updated the mock arrangement from <code>mockOfficial.setOfficialId(101)</code> to use the valid Lombok-generated <code>setUserId(101)</code> method.	<code>AppointmentServiceTest.java</code>	Test compiled and executed successfully.